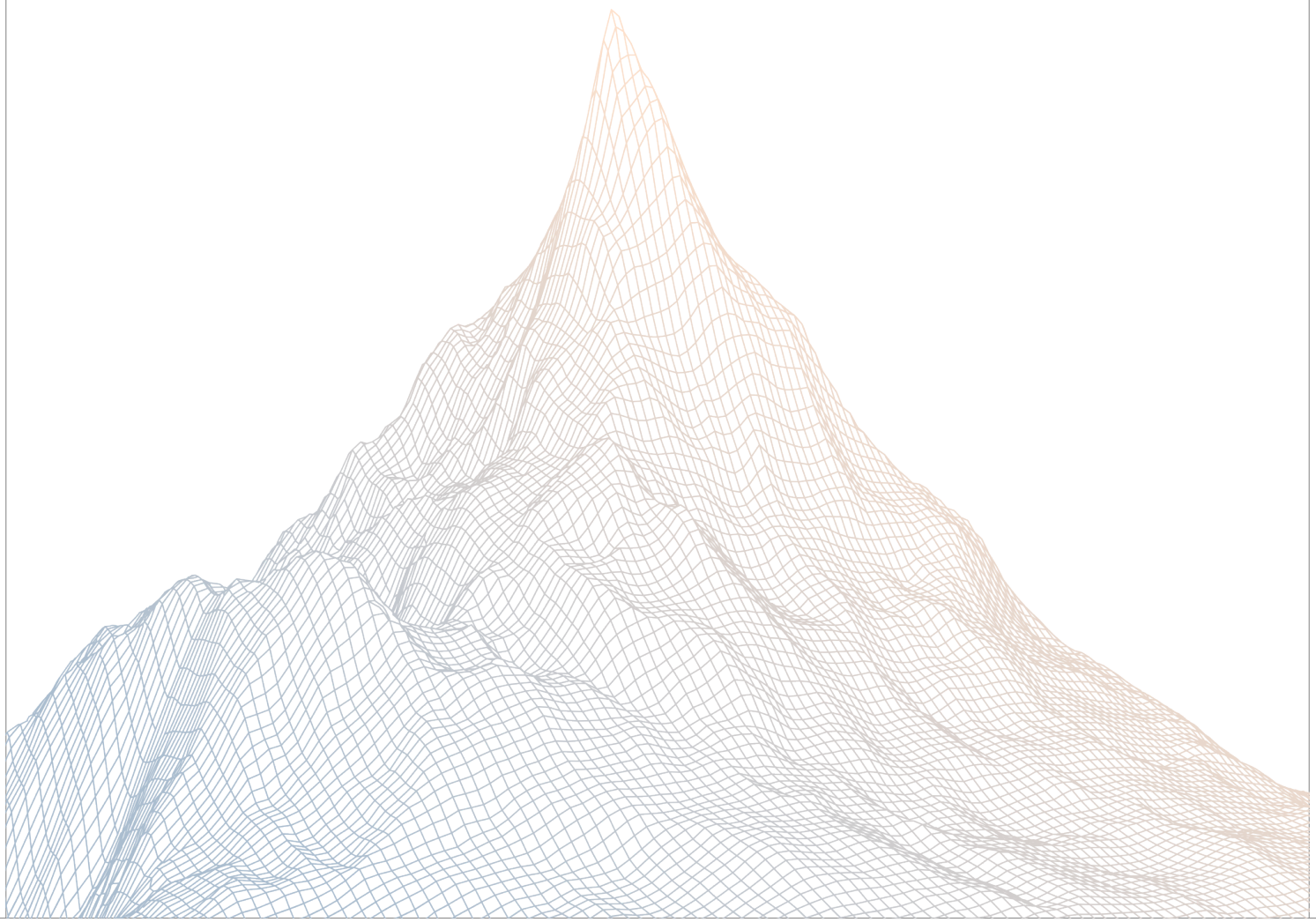
Zenith

Totalis

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: September 10th to 17th, 2026

AUDITED BY: 3doc
oakcobalt

Contents

1 Introduction 3

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 4

2.1 About Totalis 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 6

4 Findings 8

4.1 High Risk 8

4.2 Medium Risk 11

4.3 Low Risk 20

4.4 Informational 29

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Totalis

Totalis is building the derivatives layer for prediction markets. Our platform turns individual markets into building blocks that traders can combine into a single strategy. Our RFQ exchange lets market makers price each combination with greater capital efficiency, giving traders better pricing and more flexible ways to trade.

2.2 Scope

The engagement involved a review of the following targets:

Target	hyperevm
Repository	https://github.com/totalistrading/hyperevm
Commit Hash	`b92028eda62983101a885f55960996c07b7a9372`
Files	`src/**/*.sol`

Target	hyperevm Mitigation Review
Repository	https://github.com/totalistrading/hyperevm
Commit Hash	`c800ac2ec140f00525ceb4c51a2e4ef03eedbf01`
Files	`src/**/*.sol`

2.3 Audit Timeline

September 10, 2026	Audit start
September 17, 2026	Audit end
September 21, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	4
Low Risk	4
Informational	7
Total Issues	**16**

3

Findings Summary

ID	Description	Status
H-1	Public attestations can restore maker discount after registry rotation	Resolved
M-1	A chain-ID change permanently strands CoreExit-parked funds	Acknowledged
M-2	Parked CoreExit USDC freezes for gasless EOAs and certain ERC-1271 accounts	Resolved
M-3	Front-running depositMaker: a maker's deposit can be stolen via a predicted makerId	Resolved
M-4	Verifiable already-dead tickets cannot be closed, freezing stake indefinitely	Resolved
L-1	Deposit to account == address(this) will cause funds lock; sync doesn't recover	Resolved
L-2	Paying yourself to address(this) burns the caller's balance into protocol fees	Resolved
L-3	Signed withdrawals are checked for time-left, not age — so a future-dated one is an un-cancellable "post-dated cheque"	Acknowledged
L-4	Dirty 0x0814 decode can DoS EARLY_LOST closing	Resolved
I-1	The per-certificate discount cap is inflatable via self-dealt grossOpen	Acknowledged
I-2	ParlayLens.grade is not report-only: an oversized leg array reverts instead of reporting legsMatch = false	Resolved
I-3	NatSpec shows a stale pause label and a pre-E4C digest catalog	Resolved
I-4	Stolen registry admin can activate a signer before rotateAttestor lands	Acknowledged
I-5	Missing lower bound enforcement on a constructor-legal withdrawDelay	Resolved

ID	Description	Status
I-6	Open multi-leg tickets can stay unsettled after a fast SETTLED leg is read but not cached	Resolved
I-7	REDUCE_INTERVAL can ship vacuous due to missing 50% cap enforcement	Resolved

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] Public attestations can restore maker discount after registry rotation

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- src/ParlayVault.sol#L758-L771

Description:

`attest` keys `attestNonce` by the live attestor role address and verifies the caller-supplied `(cert, sig)` against that address. The EIP-712 digest does not bind the role, so the same public ECDSA payload remains valid for any successor `AttestorRegistry` that enrolls the same key, and that successor's watermark starts at 0. Anyone who observed a still-fresh honest attestation on-chain can relay it after rotation.

```
// src/ParlayVault.sol
 address who = attestor;
▷ if (a.nonce <= attestNonce[a.makerId][who]) revert
 IParlayVault.NonceNotIncreasing();
 ...
 _requireSig(who, _attestDigest(a), sig);
```

```
// src/Envelope.sol
function attestationHash(
 Attestation calldata a
) internal pure returns (bytes32) {
 return
 keccak256(
 abi.encode(
 ATTESTATION_TYPEHASH,
 a.makerId,
 a.gross0,
```

```
  a.risk0,
  a.turnover0,
  a.nonce,
  a.signedAt,
  a.expiry
  )
  );
 }
```

`AttestorRegistry.isValidSignature` recovers the ECDSA signer from the hash and returns MAGIC if that key is enrolled and matured. A replacement registry is a new role address, can legally enroll the same or overlapping hot keys from the old registry.

Impacts:

- After `resetDiscount`, replay of an already-applied `(cert, sig)` is a raise: `discount` returns, `requirement` falls, `free[m]` rises. The maker is under-reserved again against a bound the guardian or controller just treated as untrusted.
- Same-envelope replay breaks Spec SP8's "rotation adds no replay surface". Applicable collateral requirements are no longer the ones the gates enforce.

Recommendations:

Bind the live attestor role address into the attestation typehash so a role change invalidates old `(cert, sig)` bytes. Alternatively, refuse `commitRotation` onto a verifier that would still accept signatures issued under the outgoing role.

For example:

```
// src/ParlayVault.sol
function _attestDigest(
 Attestation calldata a
) private view returns (bytes32) {
 return
 _hashTypedDataV4(
 keccak256(
 abi.encode(
 ATTESTATION_TYPEHASH,
 a.makerId,
 a.gross0,
 a.risk0,
 a.turnover0,
 a.nonce,
 a.signedAt,
 a.expiry,
 attestor //@audit bind live role; type string/typehash
```

```
  must add address attestor
  )
  )
  );
}
```

Totalis: Resolved with @90eb6744bd...

Zenith: Verified.

4.2 Medium Risk

A total of 4 medium risk findings were identified.

[M-1] A chain-ID change permanently strands CoreExit-parked funds

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ParlayVault.sol
- CoreExit.sol

Description:

An `Exit` routed to `CoreExit` can be paid into the router before it is completed: anyone relays it to `withdrawWithSig`, which moves the USDC into `CoreExit` and marks the exit consumed under a **domain-bound** digest (`_hashTypedDataV4`, `src/ParlayVault.sol:319-323`). `Envelope.exitHash` is `pure` — the chain identity comes only from the EIP-712 domain separator, and OpenZeppelin **rebuilds that separator when** *`block.chainid`* **changes** (`EIP712.sol:93-96`).

`CoreExit` stores nothing; both completion paths **re-derive** the digest from the vault's *current* separator (`toTypedDataHash(VAULT.DOMAIN_SEPARATOR(), exitHash(e))`, `CoreExit.sol:55, :82`). So once the chain ID changes while funds sit parked:

1. Pre-change, `withdrawWithSig` parked the money and set `consumed[D_old]`.
2. Post-change, `CoreExit` computes `D_new ≠ D_old`.
3. `recover` needs `consumed(D_new) == true` → reverts `NotWithdrawn` (`CoreExit.sol:84`).
4. `exitToCore` sees `!consumed(D_new)`, re-calls `withdrawWithSig(e, sig)`; the vault checks `sig` against `D_new`, but it was signed over `D_old` → `BadSignature`.

No other function can move the router's USDC. The parked amount is **permanently stranded**.

Recommendations

Make the cross-contract handshake chain-ID-independent: have the vault expose, and `CoreExit` consult, the consumed-state under the raw `Envelope.exitHash(e)` rather than the domain-bound digest (the EIP-712 signature check in `withdrawWithSig` stays domain-bound; only the lookup key changes). Equivalently, have `CoreExit` cache the exit's identity on first observation and drive `exitToCore/recover` off that stored record instead of re-deriving the vault digest each call.

Totalis: The mechanism is correct. We accept the risk for three reasons. A chain-ID change on a live EVM is a chain replacement, and HyperEVM has a single operator. The parked state is transient: it needs a griefer submitting directly to the vault while the bridge is paused. Our assumption ledger (C1) already treats a chain-ID change as invalidating every envelope in the system.

Zenith: Acknowledged.

[M-2] Parked CoreExit USDC freezes for gasless EOAs and certain ERC-1271 accounts

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/CoreExit.sol#L60-L91

Description:

`CoreExit.recover` returns parked USDC after a direct vault submission, but it authorizes only `msg.sender == e.account` and never verifies a signature. Vault `withdraw` has `withdrawWithSig`, which checks `e.account` through `SignatureChecker`. `recover` has no twin, so a gasless EOA and or ERC-1271 that works for other parts of the protocol where only signature validation is needed can not call to recover. For exmaple, an ERC-1271 implementing contract with similar shape as AttestorRegistry - i.e. IERC1271 over enrolled EOAs, with no execute path that can call recover.

```
// src/CoreExit.sol
function exitToCore(Exit calldata e, bytes calldata sig) external {
 ...
 if (!VAULT.consumed(digest)) {
 VAULT.withdrawWithSig(e, sig);
 } else {
▷ if (block.timestamp > e.expiry) revert RouteExpired();
 if (before < e.amount) revert MissingFunds();
 expected = before - e.amount;
 }
 ...
}

function recover(Exit calldata e, address recipient) external {
 if (e.to ≠ address(this)) revert WrongRecipient();
▷ if (msg.sender ≠ e.account) revert Unauthorized();
 if (recipient == address(0)) revert ZeroAddress();
 if (recipient == address(this)) revert WrongRecipient();
 ...
 if (!VAULT.consumed(digest)) revert NotWithdrawn();
 ...
```

```
 SafeERC20.safeTransfer(USDC, recipient, e.amount);
}
```

```
// src/ParlayVault.sol
function withdraw(address to, uint128 amount) external nonReentrant {
 ...
 _payOut(msg.sender, to, amount);
}

function withdrawWithSig(Exit calldata e,
 bytes calldata sig) external nonReentrant {
 ...
 _requireSig(e.account, digest, sig);
 _consume(digest);
 _payOut(e.account, e.to, e.amount);
}
```

Impacts:

- A gasless EOA taker (R10: takers hold no HYPE) cannot originate `recover` until they acquire gas.
- For ceraint ERC-1271 types, the parked e.amount USDC has no signature-relayable recover and can be locked.

Recommendations:

(1) Add a signature-relayable recover path that binds `recipient` to a digest `e.account` signs. Keep existing `recover` as an optional `msg.sender == e.account` shortcut. **For example:**

```
// src/CoreExit.sol
function recoverWithSig(Exit calldata e, address recipient,
 bytes calldata sig) external {
 if (e.to ≠ address(this)) revert WrongRecipient();
 if (recipient == address(0) || recipient == address(this)) revert
 WrongRecipient();
 bytes32 digest = MessageHashUtils.toTypedDataHash(
 VAULT.DOMAIN_SEPARATOR(),
 Envelope.exitHash(e)
 );
 bytes32 recoverDigest = MessageHashUtils.toTypedDataHash(
 VAULT.DOMAIN_SEPARATOR(),
 keccak256(abi.encode(RECOVER_TYPEHASH, digest, recipient))
 );
 //@audit verify e.account; relayer cannot pick recipient without this
 sig
```

```
  if (!SignatureChecker.isValidSignatureNowCalldata(e.account,
  recoverDigest, sig)) {
  revert Unauthorized();
  }
  ...
}
```

(2) Keep `recover(Exit, address)` unchanged for EOAs that can send a transaction.

Totalis: Resolved with @99c7d9aa8a...

Zenith: Verified.

[M-3] Front-running `depositMaker`: a maker's deposit can be stolen via a predicted `makerId`

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ParlayVault.sol

Description:

Opening a maker vault and funding it are two separate transactions. `openMakerVault` hands you an ID that is just a counter ticking up by one:

```
File: src/ParlayVault.sol
336: function _openMakerVault(address controller,
 address quoteSigner) private returns (uint64 makerId) {
337: makerId = ++makerCount;
338: Maker storage m = makers[makerId];
339: m.controller = controller;
340: m.quoteSigner = quoteSigner;
341: emit IParlayVault.MakerOpened(makerId, controller,
 quoteSigner);
342: }
```

Because the next ID is always "current count + 1," a maker who wants to open vault #42 and then deposit into it must send a second transaction, `depositMaker(42, amount)`, that names that ID up front. And `depositMaker` blindly trusts the ID — it pulls your USDC and credits it to whatever vault sits at that slot, no matter who owns it:

```
File: src/ParlayVault.sol
296: function depositMaker(uint64 makerId,
 uint128 amount) external nonReentrant {
297: _requireNotPaused();
298: if (amount == 0) revert IParlayVault.ZeroAmount();
299: Maker storage m = _maker(makerId);
300: SafeERC20.safeTransferFrom(USDC, msg.sender, address(this),
 amount);
301: _creditMaker(makerId, m, msg.sender, amount);
```

```
302: }
```

The attack: Alice sends two envelopes. The first says "open me a vault" (she expects to get vault #42). The second says "put my money in vault #42." A watcher in the mempool sees both, races ahead, and sends his own "open me a vault" first — so *he* becomes the owner of #42. Alice's "open" now creates #43 instead, but her deposit envelope still says #42. Her money lands in the attacker's vault, and the attacker walks away with it (later pulling it out via the normal maker-withdraw flow).

The root cause: `depositMaker` lets you name an ID but never lets you say **who you expect to own it**.

Recommendations:

Let the depositor pin down *who* they think owns the vault, and revert if reality disagrees. Add both `expectedController` and `expectedQuoteSigner` argument and check it before taking any money: now if an attacker steals the ID, the owner won't match what Alice expected, and her deposit safely reverts instead of funding a stranger — she keeps her money and simply retries with the correct ID.

An alternative that removes the race entirely: have `openMakerVault` and the first deposit happen in a single transaction, so there's no predicted-ID gap to front-run.

In addition, consider keeping `openMakerVault` permissionless, but check `controller == msg.sender`. Anyone can still open a vault; they cannot stamp another address onto MakerOpened.

Totalis: Resolved with @c800ac2ec...

- `openMakerVault(quoteSigner)` makes the caller the controller. The `controller` parameter is gone, so nobody can open a maker in another wallet's name with their own quote signer.
- `depositMaker(makerId, expectedController, amount)` reverts `ControllerMismatch` when the maker's controller differs. A racer can no longer open the predicted id under the victim's controller, so this check now protects the deposit. Our earlier objection to it no longer holds.

The same PR adds one maker per controller (`MakerExists` and a `makerOf(address)` getter). That rule is a product change, not part of this fix, but it touches the same function. `MakerOpened` is unchanged, and storage adds one mapping at the end.

`docs/decisions.md` 2026-09-23 replaces the 2026-09-18 "no change" entry.

Zenith: Verified.

[M-4] Verifiable already-dead tickets cannot be closed, freezing stake indefinitely

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- src/ParlayVault.sol#L1386-L1412

Description:

`ParlayVault._grade` treats a pending ticket as only when a resolved selected-side `Grade.factor` is exactly 0. If that flag is false, `settle` reverts `LegPending` even when `Grade.effectiveOf` with pending slots filled at `Q` is already resovling to 0, i.e. the ticket is already lost regardless of pending legs. Unresolved slots stay hanging even when ticket results is verifiable, so an already-decided LOSS cannot be closed.

```
// src/ParlayVault.sol
 for (uint256 i; i < legs.length; ++i) {
 (bool resolved, uint64 value) = _resolve(legs[i].outcomeId,
 readGas);
 if (!resolved) {
 pending = true;
 continue;
 }
 uint64 factor = Grade.factor(value, legs[i].side);
▷ if (factor == 0) zero = true;
 factors[i] = factor;
 }

 if (pending) {
▷ if (!zero) revert IParlayVault.LegPending();
 return (SettleKind.EARLY_LOST, 0);
 }
 effective = Grade.effectiveOf(payout, factors);
 kind = Grade.kindOf(effective, stake);
```

`kindOf` runs only after every leg is resolved. It already maps `effective == 0` to `LOSS` with no factor at 0 (five legs at `v_i = 1`).

It manifests when all of:

- at least one unresolved sibling (hanging, or pruned before cache)
- no resolved selected-side factor of 0
- `floor(payout * prod(resolved_factors) * Q^pending / Q^n) == 0`

A deployer can settle one leg to `1` instead of `0` and leave a sibling unset. A sibling can also simply never settle.

Impacts:

- Permissionless `settle` cannot close an already-decided LOSS (`LegPending`).
- A hanging or pruned-uncached sibling then permanently freezes both sides of the ticket notional: taker stake in `stakeEscrow` (the taker's claim is already 0; they still cannot exit it) and maker `worst0` in `grossOpen` / `requirement`, plus the won stake that should have become maker-free on `LOSS`.
- In case of one of the leg stay permantly unsettled, `EARLY_LOST` provide escape, but same verifiable outcome tickets stay hanging. In both cases, taker has no econmic incentives to cooperate with maker for sellback.

Recommendations:

In the `_grade` pending branch, return `EARLY_LOST` when `effectiveOf(payout, factors_with_pending_as_Q) == 0`. Accounting already matches `LOSS` at `effective == 0`.

For example:

```
// src/ParlayVault.sol
 if (pending) {
 if (!zero) {
 uint64[] memory optimistic = factors;
 for (uint256 i; i < optimistic.length; ++i) {
 if (optimistic[i] == 0) optimistic[i]
 = Grade.Q; //@audit fill pending slots at Q
 }
 if (Grade.effectiveOf(payout, optimistic) == 0) {
 return (SettleKind.EARLY_LOST,
 0); //@audit already dead under optimistic Q
 }
 revert IParlayVault.LegPending();
 }
 return (SettleKind.EARLY_LOST, 0);
 }
```

Totalis: Resolved with @7764b098bd...

Zenith: Verified.

4.3 Low Risk

A total of 4 low risk findings were identified.

[L-1] Deposit to `account = address(this)` will cause funds lock; sync doesn't recover

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/ParlayVault.sol#L272-L294

Description:

`depositUser` and `depositUserWithAuthorization` reject only `account = address(0)`. A credit to `account = address(this)` still pulls USDC and runs `_creditUser`, so `lanesTotal` and `balance[address(this)]` both increase.

```
// src/ParlayVault.sol
function depositUser(
 address account,
 uint128 amount
) external nonReentrant {
 _requireNotPaused();
 if (account == address(0)) revert IParlayVault.ZeroAddress();
 if (amount == 0) revert IParlayVault.ZeroAmount();
 SafeERC20.safeTransferFrom(USDC, msg.sender, address(this), amount);
 _creditUser(account, msg.sender, amount);
}
```

A caller sets `account` to `address(this)` on either deposit, from a UI or relayer mistake. `sync` treats `USDC.balanceOf(address(this)) - lanesTotal` as surplus and reverts `NoSurplus` , so the matching credit is not swept to treasury.

Recommendations:

Reject `account == address(this)` in `depositUser` and `depositUserWithAuthorization`.

For example:

```
// src/ParlayVault.sol
function depositUser(address account,
 uint128 amount) external nonReentrant {
 _requireNotPaused();
 if (account == address(0) || account == address(this)) revert
 IParlayVault.ZeroAddress(); //@audit also block self-credit
 ...
}
```

Totalis: Resolved with @8b6862d01c...

Zenith: Verified.

[L-2] Paying yourself to `address(this)` burns the caller's balance into protocol fees

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ParlayVault.sol

Description:

The constructor already knows that the vault paying *itself* is dangerous — it flatly forbids `feeSink == address(this)`:

```
init.owner == init.feeSink
 || ...
 || init.feeSink == address(this) // ← self-payment is explicitly
 banned here
```

The reason: a transfer from the vault to the vault moves zero tokens, but the ledger still gets debited as if money left. That leftover amount becomes "surplus" that the permissionless `sync()` scoops into the treasury.

But the exact same self-payment is reachable — and **not** guarded — on all three money-out paths, because they happily accept `to == address(this)`:

- `withdraw(address to, ...)`
- `withdrawWithSig(Exit e, ...)` (uses `e.to`)
- `executeMakerWithdraw(...)` (uses the stored `r.to`)

Every one of them ends in the same shape: debit the ledger, then transfer:

```
File: src/ParlayVault.sol
1599: function _payOut(address account, address to,
 uint128 amount) private {
 ---
1603: balance[account] = held;
1604: lanesTotal -= amount;
1605: SafeERC20.safeTransfer(USDC, to, amount);
 ---
```

```
1608: }
```

You ask the vault to pay you, but you accidentally write the vault's *own* address as the destination. The vault crosses your money off its books ("paid!") — but since it paid itself, no coins actually left the drawer. Now the drawer holds more coins than the books say it should. Anyone can then call `sync()`, which notices the "extra" coins and hands them to the treasury:

```
File: src/ParlayVault.sol
832: function sync() external nonReentrant {
833: uint256 lanes = lanesTotal;
 ---
836: uint256 surplus = held - lanes;
 ---
839: uint128 amount = uint128(surplus);
840: treasuryBalance += amount;
 ---
843: }
```

Net effect: your own balance is silently, irreversibly converted into protocol fees. `USDC.balanceOf(vault)` never changed, but your `balance` (or a maker's `vault`) dropped, and `sync()` gifts the difference to the treasury.

Nothing is *stolen* and the protocol stays solvent — in fact the protocol *gains*. And every path requires the fund owner's own signature/authorization, so no third party can trigger this against you. The realistic cause is a **mis-set recipient from a buggy integration** (a contract that fills in its own or the vault's address by mistake), not a malicious attack. The damage lands entirely on the careless caller.

Recommendations:

Simple fix — mirror the existing *`feeSink`* guard. Reject `address(this)` as a destination on all three exits; apply it to `withdraw` (`to`), `withdrawWithSig` (`e.to`), and `executeMakerWithdraw` (`r.to` — ideally checked when the request is *created*, so a bad recipient can't sit armed).

More robust fix — assert the payout actually moved money. Instead of blocking one specific address, require that every payout reduces the vault's real balance by exactly the amount paid. That closes *any* self-referential recipient (now or in future refactors) in a single invariant; this mirrors the delta-check the deposit path already does (`_receive` verifies the balance rose by exactly `value`), giving both directions symmetric protection.

Totalis: Resolved with @8b6862d01c....

Zenith: Verified.

[L-3] Signed withdrawals are checked for time-left, not age — so a future-dated one is an un-cancellable "post-dated cheque"

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ParlayVault.sol

Description:

When someone cashes a signed withdrawal, `withdrawWithSig` only asks how much life the cheque has *left* — it never asks how long ago it was signed:

```
File: src/ParlayVault.sol
314: function withdrawWithSig(Exit calldata e,
 bytes calldata sig) external nonReentrant {
315: _checkWindow(e.expiry, MAX_EXIT_LIFE);
```

```
File: src/ParlayVault.sol
1157: function _checkWindow(uint64 expiry,
 uint32 maxLife) private view {
1158: if (expiry < block.timestamp) revert IParlayVault.Expired();
1159: if (expiry > block.timestamp + maxLife) revert
 IParlayVault.LifeTooLong();
1160: }
```

The `Exit` envelope itself carries no signing timestamp — just an `expiry`:

```
File: src/interface/Types.sol
72: struct Exit {
73: address account;
74: address to;
75: uint128 amount;
76: uint64 expiry;
77: uint256 salt;
78: }
```

So the only rule is "`expiry` must be within the next hour (`MAX_EXIT_LIFE`)." Think of a post-dated cheque with a far-future expiry: cash it today and the teller rejects it (too far out). But it isn't torn up — it just sleeps. Months later, in the 1-hour window right before that expiry, the very same cheque becomes valid and pays out **whatever balance has piled up by then**. There is no `cancelExit`, so the signer cannot rip it up in the meantime.

Scope — nuisance, not theft. The money can only ever go to the `to` address baked in at signing, so this is not a redirection or fund-stealing vector. The harm is narrower: a stale, un-revocable payout firing at a moment the signer never intended, to a destination they may no longer want.

Who is actually exposed. A smart-contract (ERC-1271) account can effectively cancel the cheque by rotating its signer — the signature is re-validated live when the exit lands, so the old one goes dead. A plain EOA cannot: its address *is* its key, so the signature stays valid right up to `expiry`. Its only "defence" is keeping its balance below `amount` — which fails in practice, because settlement and claims credit the balance automatically.

The contract already knows how to fix this. The risk-engine certificate binds *signing age* exactly where it matters — it rejects anything older than `ATTEST_MAX_AGE`:

```
File: src/ParlayVault.sol
739: function attest(Attestation calldata a,
 bytes calldata sig) external {
 ---
743: if (a.signedAt > block.timestamp
 || block.timestamp - a.signedAt > ATTEST_MAX_AGE) {
744: revert IParlayVault.StaleAttestation();
745: }
746: if (a.expiry < a.signedAt
 || a.expiry < block.timestamp) revert IParlayVault.Expired();
```

`Attestation` carries a `signedAt`; `Exit` simply doesn't — the same discipline just isn't applied to the withdrawal envelope.

Recommendations:

Bound total life, not just remaining life. Add a `signedAt` field to `Exit` and, in `withdrawWithSig`, require `signedAt ⩽ now` and `expiry - signedAt ⩽ MAX_EXIT_LIFE`. This mirrors the attestation pattern above and turns `MAX_EXIT_LIFE` into a genuine cap on how old a cheque can be, not just how far ahead its expiry sits.

Alternatively (or additionally), make it revocable. Add a `cancelExit(bytes32 digest)` callable by `e.account` that marks the digest consumed, so a signer can burn an un-cashed withdrawal on demand.

Note that **EOA accounts are the exposed class** — ERC-1271 accounts can already self-mitigate today by rotating their authorized signer, since the signature is re-checked when

the exit lands.

Totalis: Acknowledged, not fixed. Reasoning recorded in @99c7d9aa8a....

Zenith: Acknowledged.

[L-4] Dirty 0x0814 decode can DoS EARLY_LOST closing

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/OutcomeRead.sol#L30-L50

Description:

`OutcomeRead.read` is supposed to fail only by returning `ok = false`. After a successful `staticcall` with `rd.length ⩾ 96`, uncaught `abi.decode(rd, (uint8, uint64, uint32))` can still revert on dirty high bits. That abort never reaches `_resolve`'s `(false, 0)` path, so `_grade` cannot take `EARLY_LOST`.

```
// src/OutcomeRead.sol
 /// @notice One guarded read. `ok = false` is the only failure signal.
 ...
 function read(uint32 outcomeId, uint256 readGas)
 internal
 view
 returns (bool ok, uint8 status, uint64 value, uint32 question)
 {
 (bool success, bytes memory rd) =
 OUTCOME_STATUS.staticcall{gas: readGas == 0 ? READ_GAS_DEFAULT
 : readGas}(abi.encode(outcomeId));
 if (!success || rd.length < 96) return (false, 0, 0, 0);
▷ (status, value, question) = abi.decode(rd, (uint8, uint64, uint32));
 ok = true;
 }
```

`settle` depends on that miss staying a return. `_resolve` maps `!ok` or a non-SETTLED status to `(false, 0)`. `_grade` then continues the loop and, if another leg is already a zero, closes `EARLY_LOST` instead of reverting.

```
// src/ParlayVault.sol
 (bool resolved, uint64 value) = _resolve(legs[i].outcomeId,
 readGas);
 if (!resolved) {
```

```
pending = true;
continue;
}
...
if (pending) {
if (!zero) revert IParlayVault.LegPending();
▷ return (SettleKind.EARLY_LOST, 0);
}
```

Impacts:

- Violates the documented non-revert failure contract that `settle` and early-loss rely on.
- If it happens on a non-cached leg during `settle`, the decode revert aborts before `_grade`'s `pending && zero` branch and can DoS `EARLY_LOST` closing: the ticket stays OPEN and the maker's `worst0` stays stranded.

Recommendations:

(1) Treat a failed decode as `ok=false`: reject dirty high bits (or catch the decode) and return `(false, 0, 0, 0)` so `_resolve` and `_grade` stay on the documented path.

(2) Add a unit test that a 96-byte buffer with a dirty first, second, or third word returns `ok=false` rather than reverting, and keep NatSpec aligned with that contract.

Totalis: Resolved with @7764b098bd...

Zenith: Verified.

4.4 Informational

A total of 7 informational findings were identified.

[I-1] The per-certificate discount cap is inflatable via self-dealt `grossOpen`

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ParlayVault.sol

Description:

The per-certificate discount raise is capped against the maker's *current* `grossOpen` (`_install`, `src/ParlayVault.sol:783-808`): `cap = discount + grossOpen * MAX_RAISE_BPS / BPS`. `grossOpen` is caller-inflatable — a maker opens self-dealt, fully-collateralized tickets to raise it — so one certificate's absolute cap grows with the padding.

The inflated discount then survives shrinking the book back, because a **full-win close does not consume discount**: `_closeLanes` decays only the *unrealized* part, which is zero on a win (`src/ParlayVault.sol:1505-1513`):

```
uint128 gross = grossOpen - worst0;
uint128 unrealized = worst0 - realizedCost; // full WIN:
 realizedCost == worst0 -> 0
uint128 next = discount > unrealized ? discount - unrealized
 : 0; // → discount unchanged
return (gross, next < gross ? next : gross); // only clamped
 to the smaller grossOpen
```

So a stolen attestor colluding with the maker's own quote signer can inflate `grossOpen` with self-dealt tickets → install in **one** certificate a discount the cap was meant to spread across many `REDUCE_INTERVALS` → settle the padding as **wins** to shed it while keeping the discount → drive `_requirement` to zero on the genuine book (`:1214`).

This is net-zero to the attacker and within accepted risk O4. The padding is the attacker's own capital cycling (minus fees); the resulting harm — under-collateralizing gen-

uine takers and haircutting their winnings — is exactly the accepted stolen-attestor boundary (`accepted-risks.md` O4: *"reduce collateral below the open book's true loss and expose takers to haircuts … no on-chain control contains it … detection, post hoc"*). It requires the padding to settle as wins: a losing padding decays the discount, and buyback-close does too (`sellBack` closes with `realizedCost == 0`).

What it changes is the **register's disposition, not the code's safety**:

- `:30` ("MITIGATED … a multi-certificate climb over real time instead of one shot") — the cap does **not** guarantee a real-time climb; inflation collapses it to ~one certificate, removing the reaction window.
- `:29` ("a stolen attestor … takes **half** the open book in one transaction") — the one-transaction figure is the **full** book once the cap is inflated, not half.

Recommendations:

Informational: no new fund loss beyond accepted O4. Correct `:29`/`:30` so the reaction window is not credited to the per-certificate cap. If a real control is wanted, cap raises against a delayed or time-weighted `grossOpen` checkpoint (or exclude exposure younger than one `REDUCE_INTERVAL` from the raise base), so freshly self-dealt gross cannot lift the cap.

Totalis: Acknowledged, no code change. Docs corrected in Resolved with PR-23, merged as - (totalistrading/hip4-contracts@5428cd98f00d8b3c61ec015aba8b8d38e7a1e69d).

Zenith: Acknowledged.

[I-2] `ParlayLens.grade` is not report-only: an oversized leg array reverts instead of reporting `legsMatch = false`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ParlayLens.sol

Description:

`ParlayLens.grade` is the keeper's "would `settle` close this, and how, without spending a transaction" view. It is designed to **answer, not throw** — `legsMatch` exists precisely so a wrong leg array is *reported*, and the grade masks are documented to tolerate any leg count ("past bit 255 the shift yields zero ... which `legsMatch` already reports", `src/ParlayLens.sol:329-332`).

That contract breaks for `legs.length > Grade.MAX_LEGS`. `grade` computes `legsMatch` but does **not** short-circuit on it; when every provided leg reads terminal (`pendingMask == 0`) it calls into `Grade.effectiveOf` regardless (`src/ParlayLens.sol:101`), and `effectiveOf` reverts on the very input the Lens is supposed to classify (`src/Grade.sol:58`):

```
if (n > MAX_LEGS) revert IParlayVault.BadLegCount();
```

So a caller (keeper/monitor) that batches or auto-retries over user-supplied leg arrays gets an opaque `BadLegCount` revert instead of `legsMatch = false, settleable = false`. It is also inconsistent: the same oversized array returns cleanly when one of its legs happens to be pending (the `pendingMask ≠ 0` early return at `:99`), and only reverts when all legs read terminal.

Recommendations:

Short-circuit before `Grade.effectiveOf`: if `!g.legsMatch` (or `legs.length > Grade.MAX_LEGS`), return the populated `legsMatch/pendingMask/cachedMask` fields with `settleable = false` rather than entering `effectiveOf`. This keeps `grade` total over every caller-supplied leg array, matching its documented "report, don't revert" contract.

Totalis: Resolved with @7764b098bd....

Zenith: Verified.

[I-3] NatSpec shows a stale pause label and a pre-E4C digest catalog

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/ParlayVault.sol#L434-L436

Description:

NatSpec in scoped src still describes a pre-E4C digest catalog and marks the ticket-lifecycle section PAUSABLE. The same contracts already ship `cashoutTransferDigest` / E4C and document `cancelQuote`, `settle`, `cacheSettlement`, and `claim` as NEVER PAUSABLE on their own notices.

```
// src/ParlayVault.sol
 //
 // ------------------------------------------------------------------------
 // --
▷ // Ticket lifecycle -- spec.md 2, row 12.  PAUSABLE.
 //
 // ------------------------------------------------------------------------
 // --

 function accept(...) external nonReentrant {  ...  }

 /// @notice Row 12C.  ...  NEVER PAUSABLE and never retirement-gated  ...
 function cancelQuote(Quote calldata q) external {  ...  }

 /// @notice Row 13.  Permissionless, NEVER PAUSABLE: settlement is the
 /// taker's exit from risk and the maker's release of `worst0` (M21).
 function settle(...) external nonReentrant {  ...  }
```

`IParlayLens` still says five digest views. `cashoutTransferDigest` is a sixth, and `digestConsumed` / G9 `consumed` still name only E3/E4/E5. `Types.sol` pins E3/E4/E5 on sibling structs and not E4C on `CashoutTransfer`. `OutcomeRead`'s library notice names `accept` and `settle` as the write-path `0x0814` readers and omits `sellBack` and `cashoutTransferTicket` (via `legStateOf`).

```
// src/interface/IParlayLens.sol
/// @notice Stateless: every view takes the vault and ECHOES it back, so a
/// consumer wired to a run-off generation cannot mistake it for the live
/// one
▷ /// (spec.md 5, SP17). The five digest views are the exception -- an
 EIP-712
/// digest already contains `verifyingContract`.
interface IParlayLens {
 ...
 function cashoutTransferDigest(address vault,
 CashoutTransfer calldata t) external view returns (bytes32);

▷ /// @notice The one-shot map for E3/E4/E5 (SP2).
 function digestConsumed(address vault,
 bytes32 digest) external view returns (address vaultEcho,
 bool consumed);
}
```

Recommendations:

(1) Narrow the `ParlayVault` ticket-lifecycle section header so PAUSABLE applies only to row 12 (`accept`), or split never-pausable rows into their own block.

(2) Update scoped NatSpec to the post-E4C surface: six digest views; `consumed/digestConsumed` cover E3/E4/E4C/E5; cite E4C on the `CashoutTransfer` struct; name `sellBack/cashoutTransferTicket` (via `legStateOf`) alongside `accept` and `settle` in `OutcomeRead` if that notice is a write-path catalog.

For example:

```
// src/interface/IParlayLens.sol
/// The six digest views are the exception -- an EIP-712 digest already
/// contains `verifyingContract`.
//@audit include E4C on the one-shot map
/// @notice The one-shot map for E3/E4/E4C/E5 (SP2).
```

Totalis: Resolved with @892ad0223d...

Zenith: Verified.

[I-4] Stolen registry admin can activate a signer before rotateAttestor lands

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- src/AttestorRegistry.sol#L58-L58

Description:

`AttestorRegistry` documents that signer addition is delayed so a stolen admin cannot install a hostile key faster than the vault can rotate the attestor role. The constructor does not implement that comparison. It only rejects `addDelay == 0` or `addDelay > 90 days`. The vault bounds `ROTATE_DELAY` the same way, independently, and `commitRotation` stores a new attestor address without reading `ADD_DELAY()`.

```
// src/AttestorRegistry.sol
constructor(address admin, uint32 addDelay, address[] memory signers) {
 if (admin == address(0)) revert ZeroSigner();
▷ if (addDelay == 0 || addDelay > MAX_ADD_DELAY) revert BadDelay();
 // no compare to the vault's ROTATE_DELAY
```

`proposeSigner` then matures the key after `ADD_DELAY` only:

```
// src/AttestorRegistry.sol
function proposeSigner(address s) external {
 ...
 uint64 since = uint64(block.timestamp) + ADD_DELAY;
 activeAt[s] = since;
```

A registry deployed at birth or later with `ADD_DELAY` shorter than the live `ROTATE_DELAY` is a legal on-chain pair. After a stolen `ADMIN` calls `proposeSigner`, the key validates via ERC-1271 once `ADD_DELAY` elapses. Replacing that registry still waits out `ROTATE_DELAY`. An off-chain ceremony lint may refuse the pair at initial deploy; it is not a constructor, rotation, or runtime check.

Impact:

- Hostile signer can `attest` (never pausable) during the `ADD_DELAY..ROTATE_DELAY` gap.
- Extra loss is a shorter time-to-live-key than the documented rotation alarm, not a new money path.
- Raises stay capped by `MAX_RAISE_BPS` per `REDUCE_INTERVAL`; `pause()` and `resetDiscount` remain available.

Recommendations:

(1) In vault constructor, After `ROTATE_DELAY` and attestor are set, if enforcing the pairing on chain, read `ADD_DELAY()` on `init.attestor` and revert when it is below `ROTATE_DELAY`.

(2) On `commitRotation` of `Role.ATTESTOR` (and at vault construction if the attestor has code), read `ADD_DELAY()` and revert when it is below `ROTATE_DELAY`, so a later replacement registry cannot silently undercut the rotation alarm.

Totalis: Acknowledged. Resolved with @90eb6744bd..., in the deploy ceremony rather than on chain. script/deploy-vault.mjs preflight now verifies the runtime at init.attestor against the AttestorRegistry artifact (immutables masked), requires its live ADD_DELAY() to equal the manifest value the bounds lint already relates to rotateDelay, and requires ADMIN() to be the vault owner. Any non-throwaway manifest must declare its attestor as a registry or as an external ERC-1271 signer. AttestorRegistry NatSpec no longer implies a constructor guarantee, and the attestor rotation runbook in docs/security.md requires the same reads before any later proposeRotation to a new registry.

Zenith: Acknowledged.

[I-5] Missing lower bound enforcement on a constructor-legal withdrawDelay

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/ParlayVault.sol#L225-L234

Description:

The constructor stores `WITHDRAW_DELAY` as an immutable after rejecting only `withdrawDelay == 0` or `withdrawDelay > 30 days`. A value of 1 is accepted. The security-meaningful floor of 86400 lives only in `script/manifest-bounds.mjs`.

```
// src/ParlayVault.sol
 // Rejects only the clearly-wrong: a zero that disables a mechanism
 // the
 // design relies on ... The strict envelope
 // is the ceremony's (`script/manifest-bounds.mjs`); an immutable
 // bound
 // that binds an honest deploy forces a migration.
 if (
 init.maxRaiseBps == 0 || init.reduceInterval == 0
 || init.withdrawDelay == 0 || init.attestMaxAge == 0
 || init.rotateDelay == 0 || init.maxQuoteLife == 0
 || init.maxSellBackLife == 0
 || init.maxTransferLife == 0 || init.maxExitLife == 0
 || init.maxTicketPayout == 0
 ) revert IParlayVault.InitOutOfBand();
 if (
 init.maxRaiseBps > BPS || init.reduceInterval > 7 days
 || init.withdrawDelay > 30 days
 || init.attestMaxAge > 7 days || init.rotateDelay > 90 days
 || init.maxQuoteLife > 30 days
 || init.maxSellBackLife > 30 days
 || init.maxTransferLife > 30 days
 || init.maxExitLife > 30 days
 ) revert IParlayVault.InitOutOfBand();
```

`_requestMakerWithdraw` sets `readyAt = block.timestamp + WITHDRAW_DELAY`. `executeMakerWithdraw` pays once that timestamp is reached. `cancelMakerWithdraw` is the guardian's denial.

```
// src/ParlayVault.sol
 uint64 readyAt = uint64(block.timestamp + WITHDRAW_DELAY);
```

A deploy that skips offchain `checkBounds` can construct with `withdrawDelay` below 86400. After `requestMakerWithdraw`, `executeMakerWithdraw` can complete before a guardian monitor calls `cancelMakerWithdraw`.

Impact: WITHDRAW_DELAY is the incident window on the delayed maker-withdraw path. Spec sets it to 24 hours so a guardian can see MakerWithdrawRequested, inspect the frozen to, and cancelMakerWithdraw (or pause) before executeMakerWithdraw pays. Missing minimum guard removes sanity check guard rail for security sensitive parameter.

Recommendations:

(1) Revert `InitOutOfBand` when `withdrawDelay` is below a hard floor such as `1 hours`.

(2) Consider adding minimal threshold for other security sensitive parameters to align with specs.

Totalis: Resolved with @6652d69660...

Zenith: Verified.

[I-6] Open multi-leg tickets can stay unsettled after a fast SETTLED leg is read but not cached

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/ParlayVault.sol#L681-L696

Description:

`cashoutTransferTicket` and `sellBack` already observe a live HIP-4 SETTLED value through view `legStateOf` / `_outcomeState` and do not write `settlementCache`. An early `settle` that hits `LegPending` writes that cache in `_resolve` inside `_grade` and then reverts, so `_settleTicket` that cached a leg read never runs and the write is rolled back. On a multi-leg ticket, a fast leg can be pruned before ticket-level settle, and the previously readable value is relied on keepers that are explicilty not trusted to be online.

```
// src/ParlayVault.sol
 function _outcomeState(uint32 outcomeId,
 uint256 readGas) private view returns (uint8 code, uint64 value) {
 SettlementSlot storage slot = settlementCache[outcomeId];
 if (slot.set) return (STATE_TERMINAL, slot.value);

 (bool ok, uint8 status, uint64 raw,) = OutcomeRead.read(outcomeId,
 readGas);
 if (!ok) return (STATE_UNAVAILABLE, 0);
|> if (status == OutcomeRead.SETTLED) return (STATE_TERMINAL,
 Grade.clamped(raw));
 if (status == OutcomeRead.ACTIVE) return (STATE_ACTIVE, 0);
 if (status == OutcomeRead.PRUNED) return (STATE_PRUNED, 0);
 return (STATE_UNAVAILABLE, 0);
 }
```

```
// src/ParlayVault.sol
 function cashoutTransferTicket(...) external nonReentrant {
 ...
 if (Envelope.legsHash(legs) != t.legsHash) revert
 IParlayVault.LegsMismatch();
```

```
|> if (legStateOf(legs, readGas) != x.legState) revert
 IParlayVault.LegStateMismatch();
 _consume(digest);
 _transferTicket(t, x.ticketId, x.seller, x.buyer, x.price);
 }
```

```
// src/ParlayVault.sol
 /// @dev When the ticket cannot close the call reverts atomically, cache
 /// writes included -- `cacheSettlement` is the keeper's tool, not a side
 /// effect of a failed settle (M9).
 function settle(...) external nonReentrant {
 (SettleKind kind, uint128 effective) = _grade(t.payout, t.stake,
 legs, readGas);
 _settleTicket(ticketId, t, kind, effective);
 }
```

```
// src/ParlayVault.sol
 (bool resolved, uint64 value) = _resolve(legs[i].outcomeId,
 readGas);
 if (!resolved) {
 pending = true;
 continue;
 }
 ...
 if (pending) {
|> if (!zero) revert IParlayVault.LegPending();
 return (SettleKind.EARLY_LOST, 0);
 }
```

A ticket with staggered HIP-4 resolution is the load-bearing case: a fast leg is SETTLED while a needed sibling is still non-terminal and no resolved factor is 0. Keeper-offline is in scope, so `cacheSettlement` may not run in that window. After the fast leg is PRUNED, later `settle` cannot recover the earlier SETTLED value. **Impacts:**

- The ticket may remain OPEN with no unilateral settlement once the previously readable fast-leg value is gone.
- Maker `stakeEscrow` and `grossOpen` stay locked, so later `accept` and `executeMakerWithdraw` still include the frozen ticket in coverage.
- Mutual `sellBack` remains landable but only against `STATE_PRUNED` instead of a real settled value;

Recommendations:

(1) Persist `settlementCache` on still-open paths that already read a SETTLED leg: `cashoutTransferTicket` (ticket stays OPEN), `sellBack` (closes that ticket; cache still helps other OPEN tickets on the same outcome).

For example:

```
// src/ParlayVault.sol
function _cacheSettledLegs(Leg[] calldata legs, uint256 readGas) private {
 for (uint256 i; i < legs.length; ++i) {
 //@audit persist live SETTLED reads; ignore still-pending siblings
 _resolve(legs[i].outcomeId, readGas);
 }
}
```

(2) Add a gated trusted-role import for a value verified elsewhere (another vault's cache, or an off-chain attestation) after HIP-4 has pruned. That is an authoritative cache write, not a fabricated HIP-4 read.

Totalis: Resolved with @892ad0223d...

Zenith: Verified.

[I-7] REDUCE_INTERVAL can ship vacuous due to missing 50% cap enforcement

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- src/ParlayVault.sol#L219-L234

Description:

Neither the `ParlayVault` constructor nor the ceremony lint enforces the intended 50% per-certificate raise cap (`maxRaiseBps = 5_000`). Both accept `maxRaiseBps = BPS` (10_000). At that value one certificate can raise `discount` to `grossOpen`, so `REDUCE_INTERVAL` no longer rate-limits the stolen-attestor climb.

```
// src/ParlayVault.sol
 // Rejects only the clearly-wrong: a zero that disables a mechanism
 // the
 // design relies on (`reduceInterval = 0` lets one block chain
 // cap-sized raises, so the M14 gate is what makes MAX_RAISE_BPS
 // worth
 // anything), or a value no sane deployment reaches. The strict
 // envelope
 // is the ceremony's (`script/manifest-bounds.mjs`); an immutable
 // bound
 // that binds an honest deploy forces a migration.
 if (
▷ init.maxRaiseBps == 0 || init.reduceInterval == 0
 || init.withdrawDelay == 0 || init.attestMaxAge == 0
 || init.rotateDelay == 0 || init.maxQuoteLife == 0
 || init.maxSellBackLife == 0
 || init.maxTransferLife == 0 || init.maxExitLife == 0
 || init.maxTicketPayout == 0
 ) revert IParlayVault.InitOutOfBand();
 if (
▷ init.maxRaiseBps > BPS || init.reduceInterval > 7 days
 || init.withdrawDelay > 30 days
 || init.attestMaxAge > 7 days || init.rotateDelay > 90 days
 || init.maxQuoteLife > 30 days
```

```
  || init.maxSellBackLife > 30 days
  || init.maxTransferLife > 30 days
  || init.maxExitLife > 30 days
  ) revert IParlayVault.InitOutOfBand();
```

The ceremony file claims to be tighter than that constructor ceiling, but `BOUNDS.maxRaiseBps` is still `[1, BPS].lossBps()` then computes `instantBps` from `k` only, with a hardcoded `5_000`, so a 100% cap can still acknowledge `instantBps: 5000` and pass.

```
// script/manifest-bounds.mjs
export const BOUNDS = {
  ...
 maxRaiseBps: [1n, BPS],
  ...
};
```

A deploy or ceremony that sets `maxRaiseBps` equal to `BPS` is accepted on both sides. **Impacts:** `REDUCE_INTERVAL` no longer rate-limits a stolen attestor: one certificate can raise `discount` to the full book.

Recommendations:

Reject `maxRaiseBps == BPS` (or cap strictly below `BPS`, e.g. at the intended 5_000) in both the constructor and the ceremony lint, so `REDUCE_INTERVAL` cannot ship as a no-op.

Totalis: Resolved with @6652d69660...

Zenith: Verified.